
Efficient Implementation of a Conjugate Gradient Solver on GPU

Sébastien Devaud

Under the supervision : Pof. Colin Jones and Shaoi Yang

Abstract

This paper presents a comparative study of direct and iterative methods for solving linear systems of the form $Ax = b$ in the context of MPC, where the problem dimensions are relatively small. A benchmark shows that direct methods, in particular Cholesky factorization as used in modern solvers such as NumPy or Eigen, generally provide the best performance and high accuracy.

We then study a PCG solver on NVIDIA GPUs, based on the implementation of Shaoyi Yang, and propose several improvements. These include increasing the size of the solvable problems through a more extensive use of shared memory, as well as introducing warp-level shuffle reductions to better exploit parallelism. While these optimizations improve certain memory-related aspects, the overall runtime performance remains inferior to Cholesky factorization, mainly due to memory access overheads and shared memory bank conflicts. An attempt to extend the implementation to a multi-cluster execution model was also investigated but could not be realized due to hardware compatibility issues.

Finally, we analyze the approach proposed by Alex Dao, which assigns one MPC problem per SM, enabling the concurrent solution of multiple MPC problems on the GPU. This strategy efficiently utilizes computational resources but suffers from strong pressure on the local memory of each SM. Overall, this work highlights the trade-offs between performance, parallelism, and memory constraints when solving MPC problems on GPUs.

1. Introduction

This semester project is part of the evaluation and optimization of numerical solvers executed on GPUs. The main objective was to perform benchmarks based on the code developed by Shaohui Yang, which is based on an implementation from the A2R laboratory of Dartmouth College

(Canada), available online¹. This code implements a solver based on the Preconditioned Conjugate Gradient (PCG) method for MPC applications. In addition to this code, we also test the code of Alex Du².

In a second step, improvements to the implementation of Shaohui Yang were proposed. We first attempted to fix a memory size issue, which was partially achieved, and then attempted to improve the code by using more of the resources available on the GPU.

All modifications and the benchmark scripts allowing the reproduction of the results presented in this report are available in the project GitHub repository³.

2. Background on GPU and CUDA

This section presents a general introduction to GPUs as well as to the CUDA library developed by NVIDIA. It is based on the following presentation video (Branch Education, 2024) as well as the NVIDIA documentation (NVIDIA, 2024).

2.1. Difference between GPU and CPU

From a conceptual point of view, GPUs can be compared to maritime cargo ships: they are designed to process very large amounts of data simultaneously, at the cost of higher latency. In contrast, CPUs are more similar to supersonic airplanes, capable of executing a more limited number of operations, but with significantly lower latency. This fundamental architectural difference is illustrated in Figure 1.

2.2. Memory Hierarchy of GPU

L2 cache and global memory: A GPU relies on a multi-level memory hierarchy, characterized by trade-offs between latency and capacity. Global memory is the largest but also the slowest; it is mainly used for data transfers between the host and the GPU. Data are then cached in L2 memory, which is faster and has lower capacity, from which

¹<https://github.com/A2R-Lab/GBD-PCG/tree/yang>

²<https://github.com/A2R-Lab/GATO>

³<https://github.com/Sdevaud/GBD-PCG/tree/sebastien>

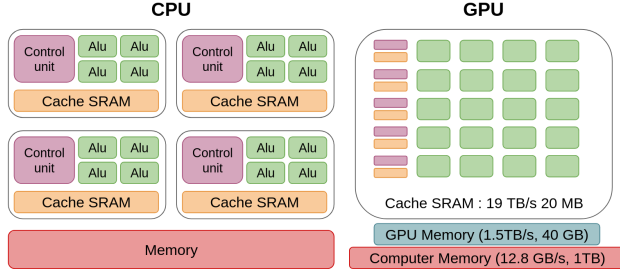


Figure 1. Conceptual Schema CPU vs GPU

most memory accesses are performed. Transfers between global memory and different regions of the L2 cache can be performed in parallel with computations on other regions, thereby improving overall throughput.

Streaming Multiprocessors: At the hardware level, a GPU is composed of several Streaming Multiprocessors (SMs). Each SM can support up to 2048 CUDA threads⁴. Threads constitute the elementary computation units⁵. Each thread has a set of internal registers, which represent the fastest memory level of the GPU, and can typically execute a fused multiply-add (FMA) operation per cycle.

Thread Blocks and Warps: Threads are grouped into sets of 32 threads called warps. An SM is then partitioned into thread blocks, each composed of several warps. The number of warps per thread block can vary from 1 to 32⁶, while the number of thread blocks per SM varies between 32 and 2 respectively, depending on the number of warps per thread block, to reach 2048 threads per SM. The number of warps per thread block must be a power of two, and a warp can never be subdivided.

Each SM has an internal shared memory, which is very fast but of limited capacity. This memory is partitioned among the thread blocks⁷. Partitioned shared memory is private to each thread block, preventing any direct communication between blocks; therefore, any inter-block communication must go through L2 cache memory. In contrast, threads within the same warp can communicate directly via their registers, without using shared memory.

L1 cache: Finally, the L1 cache serves data accesses and is private to each SM. While it is located at the same hierarchy level as shared memory, it is hardware-managed and not directly accessible by the programmer. A summary diagram

⁴this number depends on the architecture, but most GPUs support 2048 threads per SM

⁵often assimilated to CUDA cores

⁶i.e. from 32 to 1024 threads per thread block

⁷the size allocated to each thread block is configurable and does not need to be equal

of these different components is presented in Figure 2. You can find other schematic on Annexe A.

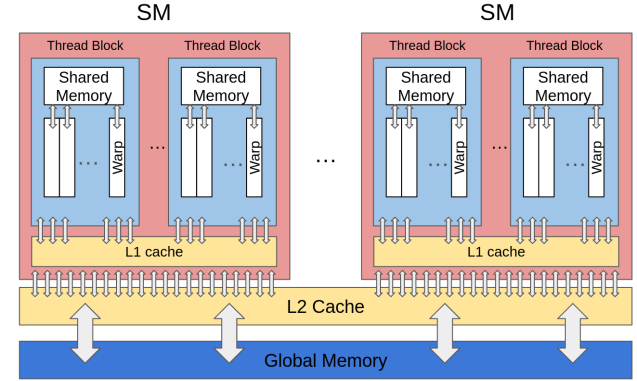


Figure 2. Conceptual Schema of the Hardware of a GPU

2.3. CUDA: a Library by NVIDIA

CUDA is a library provided by NVIDIA that allows exploiting computational resources and organizing the different memory levels of GPUs. It offers fine-grained control over shared memory, which can be manipulated at the bit level. L2 cache memory is generally not directly accessible, but recent architectures allow partial management, for example by defining read-only or write-only regions to reduce latency. Global memory only allows the management of data transfer pipelining, while L1 memory is not directly accessible to the programmer.

When developing in CUDA, a fundamental point must be taken into account: outside of a warp, any communication between threads involves a traversal up the memory hierarchy, potentially up to the L2 level. It is therefore crucial to design algorithms with few inter-warp dependencies. Moreover, only threads within the same warp share a common clock; other warps, and thus other threads, do not necessarily progress synchronously. Any extra-warp communication therefore requires explicit synchronization, which can introduce idle periods. This constraint is even more pronounced when synchronizations occur at the SM level, or at the scale of the entire GPU.

3. Mathematical Background

3.1. Model Predictive Control

We consider a MPC problem, potentially nonlinear, possibly of MIMO type, with an arbitrary cost function. We assume that the problem admits a solution, and we define the model as follows:

$$x_{k+1} = Ax_k + Bu_k, \quad k = 0, \dots, N-1. \quad (1)$$

$$\begin{aligned} \min_{\{x_k, u_k\}} \quad & J = \sum_{k=0}^{N-1} \ell(x_k, u_k) + \ell_f(x_N). \\ \text{s.t.} \quad & \text{Stable system: } g_k(x_k, u_k) = 0, \\ & \text{Constraints: } h_k(x_k, u_k) \leq 0. \\ & \text{Finite horizon: } N, \\ & \text{State: } x_k \in \mathbb{R}^{n_x}, \\ & \text{Control: } u_k \in \mathbb{R}^{n_u}, \\ & \text{System matrices: } A \in \mathbb{R}^{n_x \times n_x}, \\ & \quad B \in \mathbb{R}^{n_x \times n_u}. \end{aligned} \quad (2)$$

The resolution of the MPC problem in (2) relies of a series of KKT linear systems like equation (4), regardless of optimization algorithm choices (Interior point, ADMM, augmented Lagrangian, etc...)

$$\mathcal{S} \lambda = \gamma, \quad \mathcal{S} \in \mathbb{R}^{Nn_x \times Nn_x}, \quad \gamma, \lambda \in \mathbb{R}^{Nn_x}. \quad (4)$$

The matrix $\mathcal{S}$ is symmetric positive definite (SPD). It depends on the cost function and is constant in the case of a linear problem. The vector γ depends on the constraints of the problem, while λ represents the unknown variable. In this construction, the matrix $\mathcal{S}$ has a block tridiagonal structure of the following form:

$$\mathcal{S} = \begin{bmatrix} D_1 & O_1 & & & \\ O_1^T & D_2 & O_2 & & \\ & O_2^T & D_3 & O_3 & \\ & & O_3^T & D_4 & O_4 \\ & & & O_4^T & D_5 \end{bmatrix}, \quad (5)$$

$D_k, O_k \in \mathbb{R}^{n_x \times n_x}.$

with D_k symmetric but not O_k .

3.2. Preconditioned Conjugate Gradient

Since the matrix $\mathcal{S}$ is SPD, it is possible to use a PCG-type solver. The algorithm used in this work is presented in Algorithm (1), in accordance with the reference documentation (Nocedal & Wright, 2006).

The theoretical complexity of PCG is of order $\mathcal{O}(nzk)$, where n is the dimension of the matrix, z is the number of non-zero elements per row (or column), and k is the number of iterations of the main loop. The number of iterations strongly depends on the distribution of the eigenvalues of the matrix: the more clustered they are, the faster the

convergence. In the theoretical worst case, the number of iterations is bounded by n . The role of the preconditioner is precisely to reduce this number of iterations by improving the conditioning of the problem.

Algorithm 1 Preconditioned Conjugate Gradient

Require: Initial guess x_0 , preconditioner M^{-1}

```

1:  $r_0 \leftarrow Ax_0 - b$ 
2:  $y_0 \leftarrow M^{-1}r_0$ 
3:  $p_0 \leftarrow -y_0, \quad k \leftarrow 0$ 
4: while  $r_k \neq 0$  do
5:    $\alpha_k \leftarrow \frac{r_k^T y_k}{p_k^T A p_k}$ 
6:    $x_{k+1} \leftarrow x_k + \alpha_k p_k$ 
7:    $r_{k+1} \leftarrow r_k + \alpha_k A p_k$ 
8:    $y_{k+1} \leftarrow M^{-1}r_{k+1}$ 
9:    $\beta_{k+1} \leftarrow \frac{r_{k+1}^T y_{k+1}}{r_k^T y_k}$ 
10:   $p_{k+1} \leftarrow -y_{k+1} + \beta_{k+1} p_k$ 
11:   $k \leftarrow k + 1$ 
12: end while
```

A version of CG is provided in Appendix C. In the benchmarks presented in this work, the preconditioners are constructed according to the formulation proposed in (Yang et al., 2025), defined by

$$\begin{aligned} A &= B_l - C_l = B_r - C_r = B_d - C_d, \\ G &:= B_l^{-1} + B_r^{-1} - B_d^{-1}, \\ H &:= B_l^{-1}C_l + B_r^{-1}C_r - B_d^{-1}C_d, \\ M^{-1} &:= (I + H)G. \end{aligned} \quad (6)$$

We define a method with an order-0 preconditioner when none is used, order 1 when only G is used, and order 2 when $(I + H)G$ is used.

Finally, by using the Neumann series expansion, recalled in Lemma (7), we obtain a formal expression of the inverse of the matrix A when k tends to infinity.

Lemma 3.1. *If $A = B - C$ is a convergent splitting of a nonsingular matrix A , then*

$$\begin{aligned} A^{-1} &= \lim_{k \rightarrow \infty} (I + H + H^2 + \dots + H^k) G, \\ G &= B^{-1}, \\ H &= B^{-1}C. \end{aligned} \quad (7)$$

4. Algorithms

The sparse and block-tridiagonal structure of the matrix $\mathcal{S}$, defined in (5), allows the PCG computation to be efficiently decomposed into several independent steps. In particular, the matrix–vector multiplications occurring at lines 5 and 8 of Algorithm (1) can be performed in parallel.

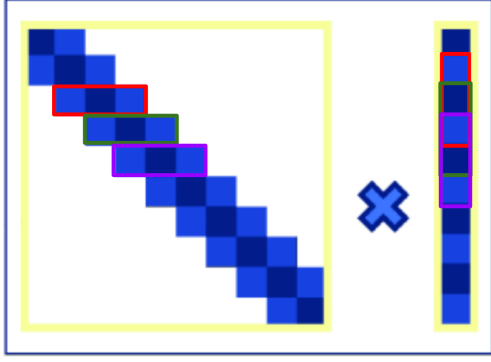


Figure 3. Conceptual Schema Matrix x Vector (Adabag et al., 2024)

Indeed, during the multiplication, each component of the resulting vector depends only on one row block of $\mathcal{S}$ of dimension n_x , and on the corresponding components of a vector of dimension $3n_x$. Each row block can therefore be processed independently of the others, which makes this operation naturally parallelizable.

A conceptual schema illustrating this block-by-block multiplication is presented in Figure (3). It should however be noted that these steps also correspond to the main bottleneck of the algorithm. Indeed, after the matrix–vector multiplication, a global reduction is required in order to compute the scalars α_k and β_k , which imposes synchronization between the different parallel computations.

In the following subsections, we focus on the two PCG implementations developed by Shaohui Yang and Alex Du.

4.1. PCG of Shaohui Yang

The implementation proposed by Shaohui Yang relies on assigning one thread block to one block row of the matrix $\mathcal{S}$. The blocks O_{i-1}^T , D_i , and O_i are loaded into shared memory in order to perform the computations there (only two blocks at the boundaries). Each thread of the thread block processes one row of the matrix, while the other SMs are used to process the other block rows.

First Problem: In the initial version, each SM was systematically divided into two thread blocks, equally sharing 42 kB of shared memory, independently of the model size and the GPU. This strategy limited the effective use of memory and imposed a maximum size. As shown in Figure (4), increasing the number of states can quickly saturate the shared memory⁸. A first improvement consisted in making the shared memory allocation dynamic, adaptable to the model size and the GPU, which makes it possible to allo-

⁸note that 96 kB is the limit for the GPU on which our tests were performed; this size differs depending on the GPU

cate the entire shared memory to a single thread block for large models. In this latter case, the algorithm reduces to assigning one SM per block row, and the number of SMs then corresponds to the horizon of the problem.

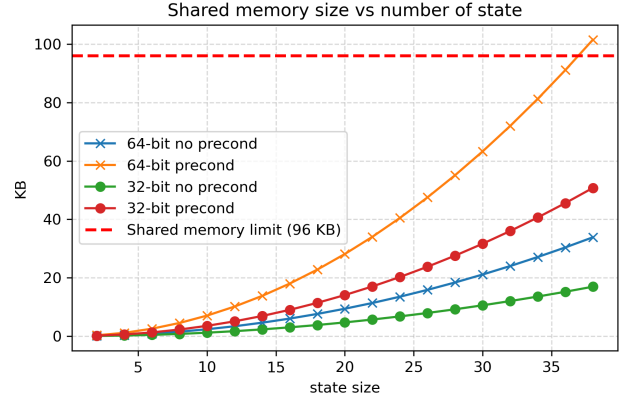


Figure 4. Size of the shared memory vs number of states

Second Problem: It concerns the under-utilization of the computing units. For example, for a number of states equal to 35, we must use the entire shared memory for one thread block, but in this case only 35 threads per SM are used, whereas 1024^9 threads are available. A considered improvement consists in assigning a matrix row not to a single thread anymore, but to a warp, by exploiting intra-warp communication capabilities to perform the matrix–vector multiplication.

Third Problem: Finally, this implementation suffers from a high number of synchronizations, both at the thread level and at the thread block and SM levels. Moreover, the private nature of shared memory leads to redundant loading of the blocks O_i^T and O_i . A potential solution would be the use of Distributed Shared Memory (DSM), introduced with thread block clusters, allowing several SMs within the same cluster to share a common memory as fast as shared memory. However, this feature is currently only available on recent architectures (Hopper, and potentially Blackwell), and requires a recent NVCC compiler¹⁰. Within the scope of this semester project, these technical constraints did not allow it to be explored experimentally.

4.2. PCG of Alex Du

The approach proposed by Alex Du relies on the use of a single SM for executing the PCG solver. Each block row of the matrix is assigned to a warp, allowing threads within

⁹the remaining 1024 threads of the other thread block cannot be used because there is no longer enough shared memory available

¹⁰compute capability ≥ 9.0 and nvcc ≥ 12.2

the same SM to communicate directly during the matrix-vector multiplication. The blocks O_{i-1}^T , D_i , and O_i are loaded into the shared memory of the thread block. When the capacity of this memory is insufficient, the additional blocks are fetched from L2 cache memory, configured as read-only.

First advantage: This approach limits data redundancy in shared memory and eliminates inter-SM synchronizations. Only intra-SM synchronizations are required, which significantly reduces the synchronization cost compared to an implementation distributed over multiple SMs.

Second advantage: Assigning one PCG problem to a single SM allows more efficient use of the available threads, while leaving the other SMs free for other tasks. This property is particularly interesting in the context of MPC, where multiple independent problems can be solved in parallel. For example, for a complex system such as a humanoid robot, where each joint can correspond to a distinct MPC problem, this implementation would allow assigning one MPC problem per SM.

Limitation: In its current state, this implementation could also benefit from the use of DSM within a cluster of one SM in order to reduce redundancy of the blocks O_{i-1}^T , D_i , and O_i . However, its main limitation lies in the frequent accesses between shared memory and L2 cache memory. Although this approach avoids the global synchronizations required in Shaohui Yang’s implementation, it remains to be determined whether this gain is sufficient to compensate for the cost of memory transfers. This question will be studied in the next section dedicated to benchmarks.

5. Benchmarks

In order to evaluate the performance and limitations of the different methods studied, several benchmarks were carried out. The tests were performed on 50 randomly generated datasets for each test; only the 90 % of the results closest to the mean were retained in order to eliminate extreme values. All tests (Python, C++, and CUDA) were performed using IEEE double/ float64 precision. The experiments were conducted on an NVIDIA GeForce RTX 5090 GPU. Additional information about this GPU, as well as other architectures, is provided in Appendix B.

For the methods of Shaohui Yang, the algorithm without preconditioner is used as is, while the *yang-precond* and *yang-optimised* versions use a preconditioner of order 2. The method of Alex (*gato* on plot) uses a preconditioner of order 1, as its implementation does not allow the use of higher-order preconditioners because in that case the matrix M^{-1} no longer has the same format. Finally, the

time required for data formatting is not taken into account in the measurements.

5.1. Benchmark with respect to the number of states

In this benchmark, the number of states is increased while the number of knot points is kept constant at 40. Figure (5) shows, on the left, the kernel execution time as a function of the number of states, and on the right, the number of iterations required for convergence for PCG-based methods. For GPU methods, only the computation time on the GPU is taken into account. The crosses correspond to individual runs, while the curves represent the average values.

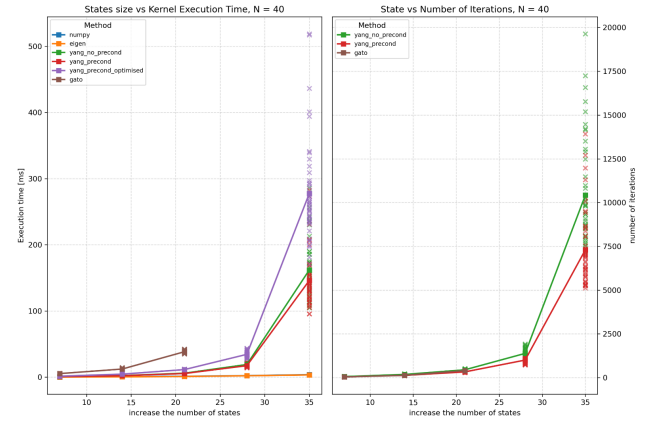


Figure 5. Increase the number of states

We first observe that the method of Alex Du does not allow the problem to be solved for a number of states greater than 21, which constitutes a major limitation.

My improvements on memory allow Yang’s algorithm to perform computations on larger model sizes, but my attempt at an optimized version does not lead to a performance gain; it is even worse, as it is slower than the base algorithm. Assigning one row to a warp did not prove to be optimal, and an approach based on assigning one block row per warp would probably have been more appropriate. However, this direction could not be further explored within the scope of the project.

An abrupt increase in computation time is observed for a number of states equal to 35. This degradation is related to a strong increase in the number of iterations, visible in the right-hand figure. Although the theoretical number of iterations is bounded by $35 \times 40 = 1400$, values far above this bound are observed in practice, due to the accumulation of numerical errors affecting the PCG stopping condition.

Overall, the execution time is strongly correlated with the number of iterations, which itself exhibits a large variance. This sensitivity to the initial data makes PCG methods less robust than Cholesky-based solvers for large models.

5.2. Benchmark with respect to the number of knot points

In this second benchmark (Figure (6)), the number of knot points is increased while the number of states is kept constant at 21. The method of Alex Du again proves to be poorly robust with respect to the model size, and once again it is the worst in terms of execution time. My attempt at optimization also remains less performant than the methods proposed by Yang, which leads us to focus only on the methods of Shaohui Yang for the remainder of the analysis.

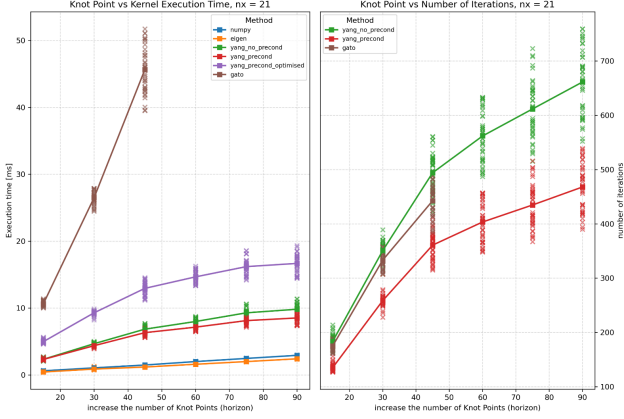


Figure 6. Increase the number of knot points

Contrary to the previous benchmark, the number of iterations required for convergence shows a lower variance with respect to the total size of the matrix S when the number of knot points increases. This observation is explained by the sparse structure of the matrix: increasing the number of knot points increases the number of blocks, but not their size, which limits the impact on PCG convergence.

However, the execution time remains sensitive to the number of iterations, but does not exhibit a sharp growth comparable to that observed when increasing the number of states. In contrast, the Eigen and NumPy solvers are more sensitive to an increase in the number of knot points, which indicates a stronger dependence on the global size of the matrix than on that of the individual blocks, even though these remain much more optimal.

5.3. Performance analysis of the proposed method

We investigated the origin of the lower-than-expected performance of the proposed shuffle-based method. From a theoretical perspective, for a system with 32 states, the reference method requires $32 \times 3 = 96$ operations, whereas the shuffle-based approach requires only $3 + \log_2(32) = 8$ operations. This significant reduction in operation count should, in principle, result in equal or superior performance.

To analyze this discrepancy, we isolated the matrix–vector

multiplication kernel and performed dedicated execution-time benchmarks. Figure (7) presents the results for four implementations, with and without shuffle operations, in both single and double precision.

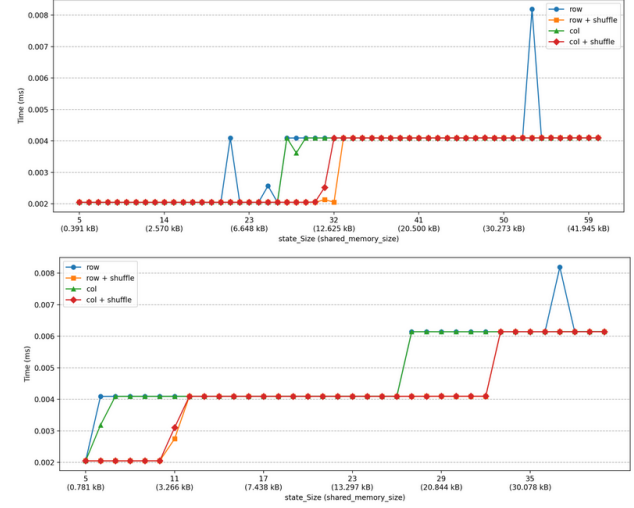


Figure 7. Matrix–vector multiplication kernel only: shuffle vs non-shuffle, row-/column-major layouts; float (top) and double (bottom). We perform 1000 run and take the average

The measurements show that execution time does not scale linearly with model size. Instead, distinct thresholds are observed: beyond certain model sizes, computation time increases sharply before reaching plateaus where further growth has no additional impact. Additionally, double-precision computations consistently incur higher execution times, even for smaller model and memory sizes.

These results indicate that performance is dominated by memory behavior rather than by arithmetic complexity. In particular, memory transfers and data layout have a significant impact, with bank conflicts¹¹ likely responsible for the observed slowdowns. Although the shuffle-based method reduces the theoretical operation count, its memory access patterns are not aligned with the existing memory organization, which is optimized for the method proposed by Shao Yang.

Consequently, the benefits of reduced computational complexity are outweighed by memory-related inefficiencies. This highlights that optimizing execution time requires prioritizing memory organization and access patterns over minimizing the theoretical number of operations.

5.4. Benchmark of the errors

This benchmark analyzes the evolution of the error for three model sizes (small, medium, and large). Figure (8) shows,

¹¹find an explanation of this problem in annex D

on the right, the error as a function of the number of iterations and, on the left, the error as a function of the execution time. The method of Alex Du is not represented in the time-based plots, as its execution time is significantly higher than that of the other methods.

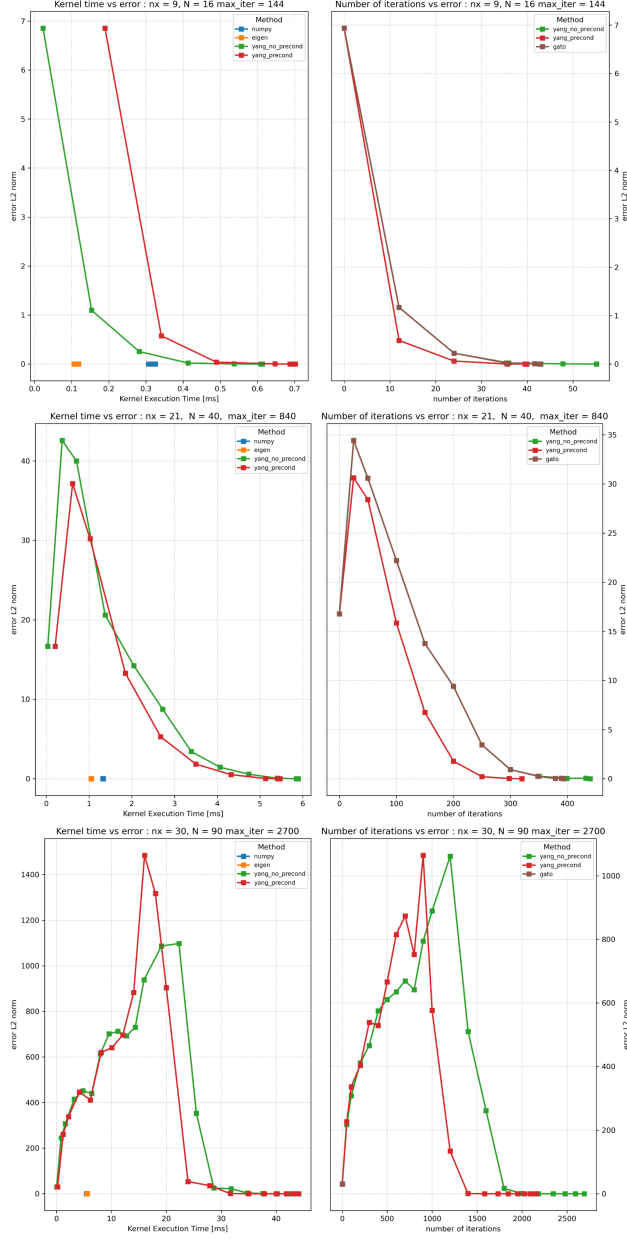


Figure 8. Evolution of the error with the number of iterations

The results show that, regardless of the model size, the PCG error does not decrease fast enough to compete with modern direct solvers. For small models, the version without preconditioner is even more time-efficient, as the additional cost related to memory accesses induced by the preconditioner, as well as the matrix–vector multiplication at line 5

of Algorithm 1, does not compensate for the lower number of iterations.

For the two other model sizes, the method of Alex Du and that of Yang without preconditioner show no significant difference (there is a small difference in error, but it is not visible in the plot), which suggests that it is preferable to use either an order-2 preconditioner or no preconditioner at all. For an order-1 preconditioner, the difference is minimal, while it unnecessarily increases the number of memory blocks as well as internal computation time.

Finally, for large models, a transient phase during which the error increases before decreasing is observed, making it difficult to define a robust stopping criterion based solely on the number of iterations. Thus, the stopping condition based on the vector r_k constitutes an important direction for future work.

5.5. Total execution time

The previous benchmarks highlighted still mixed performance for the studied algorithms, although some approaches remain promising. A major factor not taken into account so far is the data transfer time between the host (our computer) and the computing device (GPU). These transfers occur through global memory, which constitutes the slowest memory level, and can therefore have a significant impact on overall performance.

Figure (9) presents the total execution time, including data transfer to the GPU, kernel execution, and result retrieval.

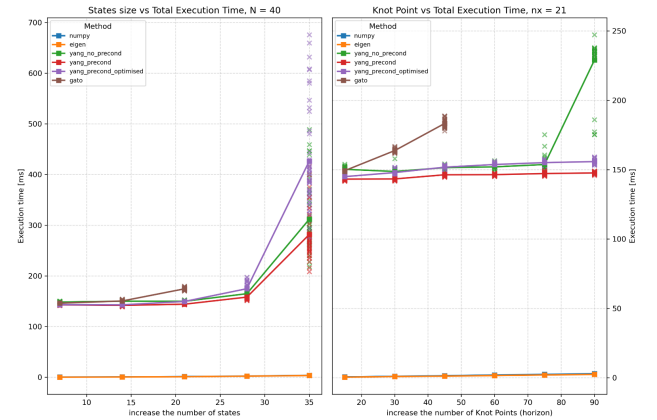


Figure 9. Total execution time for varying numbers of states and knot points

We observe that, in most cases, the transfer time is approximately 150 ms and remains relatively constant. This suggests that, given the small amount of exchanged data, only a few bus transactions are required. In contrast, in the extreme case corresponding to a number of states equal to 35, the total time is dominated by computation time, due to the

significant increase in the number of PCG iterations.

These results must however be interpreted with caution. The benchmarks were performed remotely on a server, and not on a GPU directly connected to the host machine. A more representative evaluation would require tests under local conditions. The measurements presented here are therefore indicative, but they highlight the importance of taking transfer costs into account for a GPU algorithm to outperform classical methods.

Finally, a potential optimization of the transfers would consist in avoiding redundant transmission of the matrices O_i and O_i^T . Although the available memory space is not a constraint, it would be possible to reduce the number of transfers by storing only one of the two matrices and adapting the computations accordingly.

6. Conclusion

This work has highlighted the efficiency and robustness of classical numerical solvers, such as Eigen and NumPy, for solving sparse linear systems, in particular via direct methods of the Cholesky type. In contrast, the PCG method presents an intrinsic difficulty related to the variability of the number of iterations, which strongly depends on the initial data and affects performance robustness.

The role of preconditioning proved to be central. The preconditioner proposed by Shaohui Yang significantly reduces the number of PCG iterations, but the results show that an order-1 preconditioner is often insufficient, while a higher-order preconditioner or the absence of a preconditioner can turn out to be more effective. Improving and designing suitable preconditioners therefore constitutes a priority research direction.

The obtained results also indicate that the approach of Alex Du does not, in its current form, manage to compensate for the limitations related to memory accesses and synchronizations. Frequent transfers with L2 memory degrade performance, making this implementation less efficient than that of Shaohui Yang. Nevertheless, improvement directions remain for both algorithms, in particular through better exploitation of L2 memory, the use of DSM, an increase in the number of used threads, and a reduction of redundant data transfers.

For the isolated solution of the linear system $Ax = b$, classical methods remain clearly superior, especially for small models, where execution times on the order of 0.2 microseconds can be achieved. However, the objective of this work is not limited to this problem, but concerns the complete solution of an MPC problem. In this context, additional computations are required, and the approach consisting in isolating one MPC problem per SM presents a certain conceptual

interest for complex systems requiring multiple MPC controllers simultaneously.

Finally, the use of NVIDIA's cuDSS library could not be evaluated due to software constraints. Moreover, its automatic exploitation of all GPU resources makes it poorly suited to strategies requiring fine control over SM allocation. It could also be interesting to implement a custom Cholesky solver in order to compare performance.

In conclusion, the use of a GPU coupled with a PCG solver for solving MPC problems remains promising, but currently suffers from limitations in terms of robustness and performance. This approach appears mainly relevant for large-scale models, whereas for simple systems or a single MPC problem, such as a mini-drone, a microcontroller-based implementation using classical solvers remains more efficient.

References

- Adabag, E., Atal, M., Gerard, W., and Plancher, B. Mpcgpu: Real-time nonlinear model predictive control through preconditioned conjugate gradient on the gpu. In *Proceedings of the IEEE International Conference on Robotics and Automation*, May 2024. URL <https://ieeexplore.ieee.org/document/10611212>.
- Branch Education. How do graphics cards work? exploring gpu architecture, 2024. URL <https://www.youtube.com/watch?v=h9Z4oGN89MU>.
- Nocedal, J. and Wright, S. J. *Numerical Optimization*. Springer, second edition, 2006. URL <https://www.math.uci.edu/~qnie/Publications/NumericalOptimization.pdf>.
- NVIDIA. Cuda c programming guide, chapter 6, 2024. URL <https://docs.nvidia.com/cuda/cuda-c-programming-guide/#distributed-shared-memory>.
- Technical CITY. Unbiased hardware comparisons, 2020. URL <https://technical.city/en>.
- Yang, S., Ohtsuka, T., Plancher, B., and Jones, C. N. Polynomial and parallelizable preconditioning for block tridiagonal positive definite matrices, May 2025. URL <https://arxiv.org/pdf/2503.15269>.

A. Other mind Map of GPU with Cluster

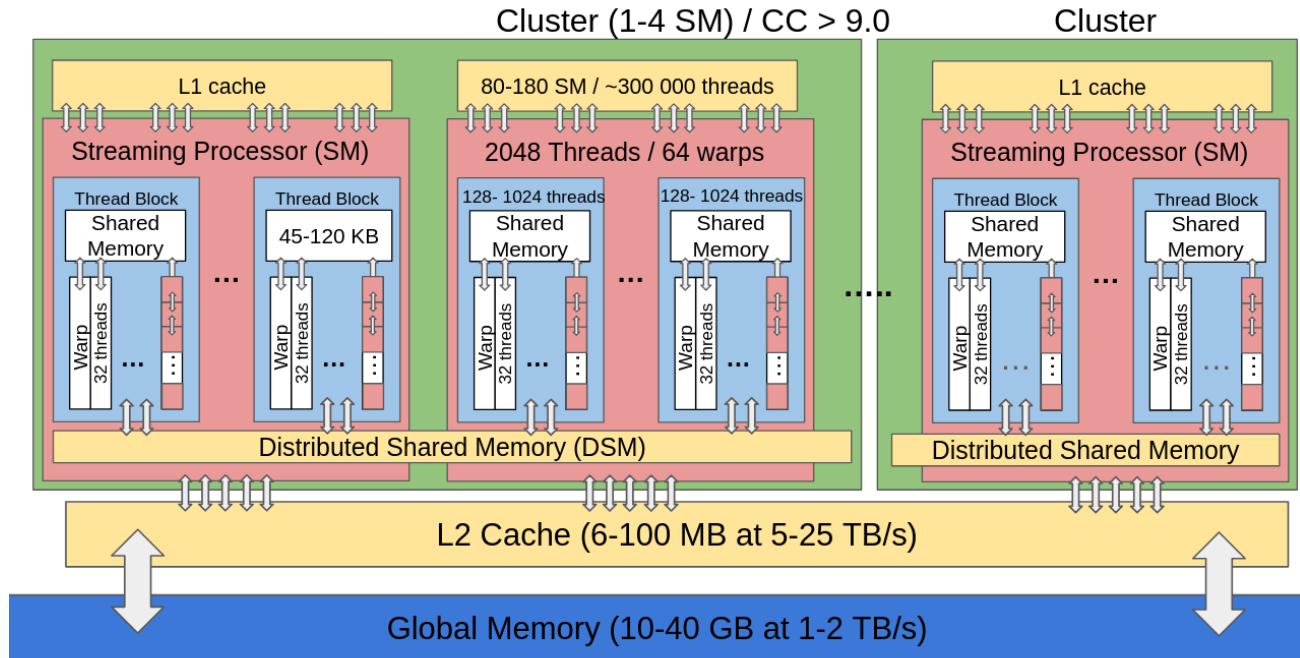


Figure 10. GPU mind map with CLuster

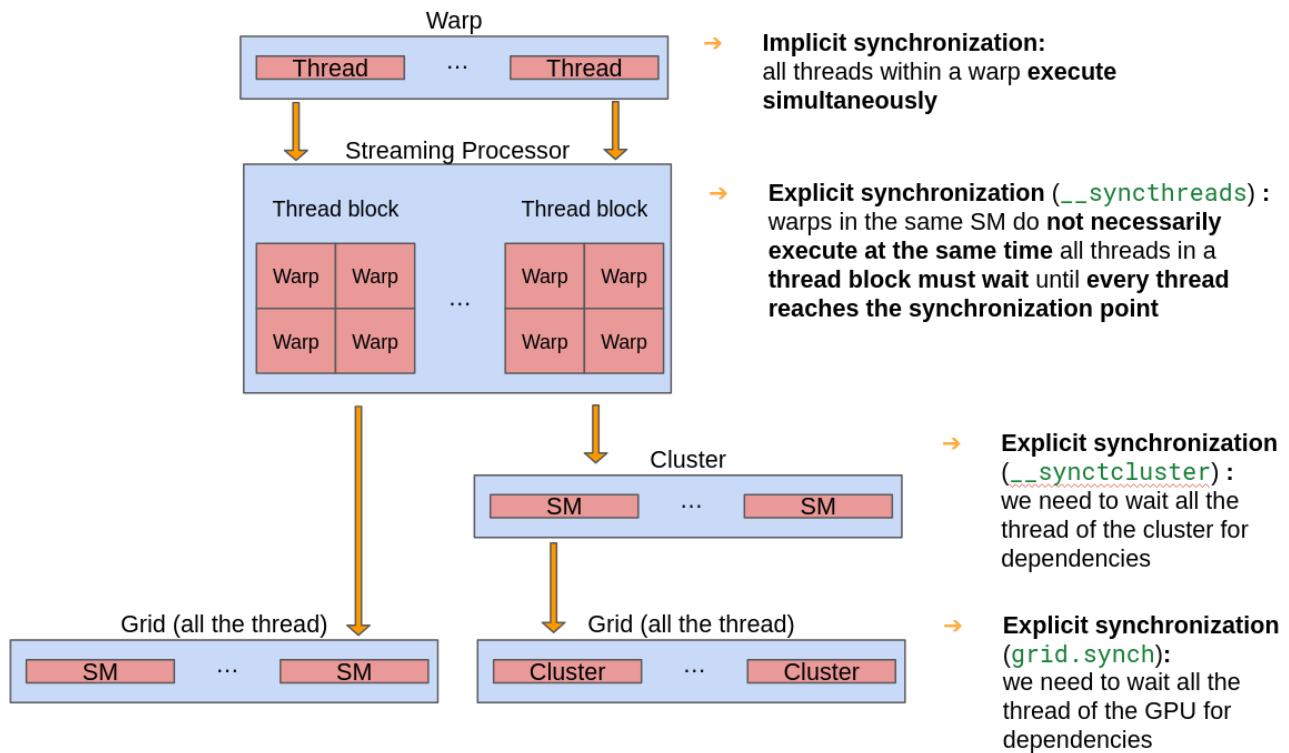


Figure 11. Synchronisation mind map

B. Characteristics of the GPUs

Table 1 presents the main characteristics of the GPUs used for the benchmarks. It highlights the rapid evolution of GPU performance over recent years, both in terms of computational power and memory capacity, features, and cost. New architectures, renewed on an annual basis, introduce significant improvements, particularly for applications related to artificial intelligence.

An important distinction must however be made between GPUs intended for data centers and those targeting the consumer market. The former are characterized by a significantly higher cost, mainly justified by their robustness and their ability to operate continuously over long periods while ensuring high computational reliability. In contrast, consumer GPUs are not designed for prolonged operation with the same guarantees of accuracy and durability.

In the context of the MPC solvers studied in this work, the use of large GPUs, both in terms of physical dimensions and memory capacity, is not optimal. The considered problems are of modest size with respect to the resources offered by modern GPUs, and the objective is integration into embedded systems. In this context, a compact graphics card, offering fast memory transfers, in particular between the computer and the GPU, is preferable to an architecture equipped with very wide buses or large memory capacities.

NVIDIA has proposed certain solutions in this direction, notably with cards such as the Jetson Orin Nano. However, the current trend in GPU development is largely oriented toward the needs of large data centers for artificial intelligence. Research and industrialization efforts therefore focus on increasing computational power and memory capacity, rather than on miniaturization of cards and optimization of computation speed for embedded applications. A small comparative table of GPU performance differences is shown below in Table (1):

Table 1. NVIDIA GPU Memory and Architecture Comparison (Technical CITY, 2020)

Parameter	Tesla V100 PCIe	GeForce RTX 5090	Jetson Orin Nano
Global memory	16 GB (HBM2)	32 GB (GDDR7)	8 GB (LPDDR5)
Global memory bandwidth	0.9 TB/s	1.79 TB/s	68.22 GB/s
L2 cache size	6 MB	96 MB	256 KB
Estimated L2 bandwidth	~5 TB/s	~25 TB/s	–
Shared memory per SM	96 KB	96 KB	64 KB
Core clock speed	1.76 GHz	1.75 GHz	0.533 GHz
Memory bus width	4096 Bit	512 Bit	128 Bit
Number of SMs	108	170	16
Total threads (2048 per SM)	218 592	348 160	32 768
CUDA compute capability	7.0	12.0	8.6
Release year	2020	2025	2023
Price	10 000 \$	2 000 \$	300 \$
Compute Capability	7.0	12.1	8.9
DSM Cluster	No	Yes	No

C. PCG without preconditioner

Here is the algorithm without the preconditioner. This version will therefore iterate much more on its main loop k . However, in this algorithm, we no longer need to perform the matrix–vector multiplication at line 8 of Algorithm 1, which is one of the most expensive operations at each cycle. This is therefore once again a trade-off, where the matrix M^{-1} needs to be well constructed; otherwise, it is preferable to use this version without a preconditioner.

Algorithm 2 Conjugate Gradient

Require: Initial guess x_0

```
1:  $r_0 \leftarrow Ax_0 - b$ 
2:  $p_0 \leftarrow -r_0, \quad k \leftarrow 0$ 
3: while  $r_k \neq 0$  do
4:    $\alpha_k \leftarrow \frac{r_k^T r_k}{p_k^T A p_k}$ 
5:    $x_{k+1} \leftarrow x_k + \alpha_k p_k$ 
6:    $r_{k+1} \leftarrow r_k + \alpha_k A p_k$ 
7:    $\beta_{k+1} \leftarrow \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$ 
8:    $p_{k+1} \leftarrow -r_{k+1} + \beta_{k+1} p_k$ 
9:    $k \leftarrow k + 1$ 
10: end while
```

D. Bank Conflict

D.1. Shared memory bank conflicts

Modern GPUs organize shared memory into multiple independent memory banks, typically one bank per thread in a warp (e.g., 32 banks for a 32-thread warp). In an ideal scenario, each thread accesses a different memory bank, allowing all memory requests within a warp to be served simultaneously in a single cycle.

Figure 12 illustrates two different access patterns. On the right-hand side, each thread in the warp accesses a distinct memory bank. As a result, all memory accesses are completed in one cycle, achieving maximum shared memory bandwidth.

In contrast, the left-hand side of the figure shows a situation where multiple threads attempt to access the same memory bank during the same cycle. Since a memory bank can only serve one access per cycle, these concurrent accesses are serialized. This phenomenon is known as a *bank conflict*. In the illustrated example, the presence of bank conflicts forces the warp to complete its memory requests over multiple cycles, leading to increased latency and reduced performance.

Bank conflicts are therefore not caused by the number of memory accesses themselves, but by the access pattern and memory layout. Even when the total number of operations is reduced, poorly aligned or non-uniform memory accesses can introduce significant slowdowns. This explains why algorithms with lower theoretical arithmetic complexity may still exhibit inferior performance if their shared memory access patterns generate bank conflicts.

Consequently, optimizing GPU performance requires careful consideration of shared memory organization and access patterns to ensure conflict-free memory accesses whenever possible.

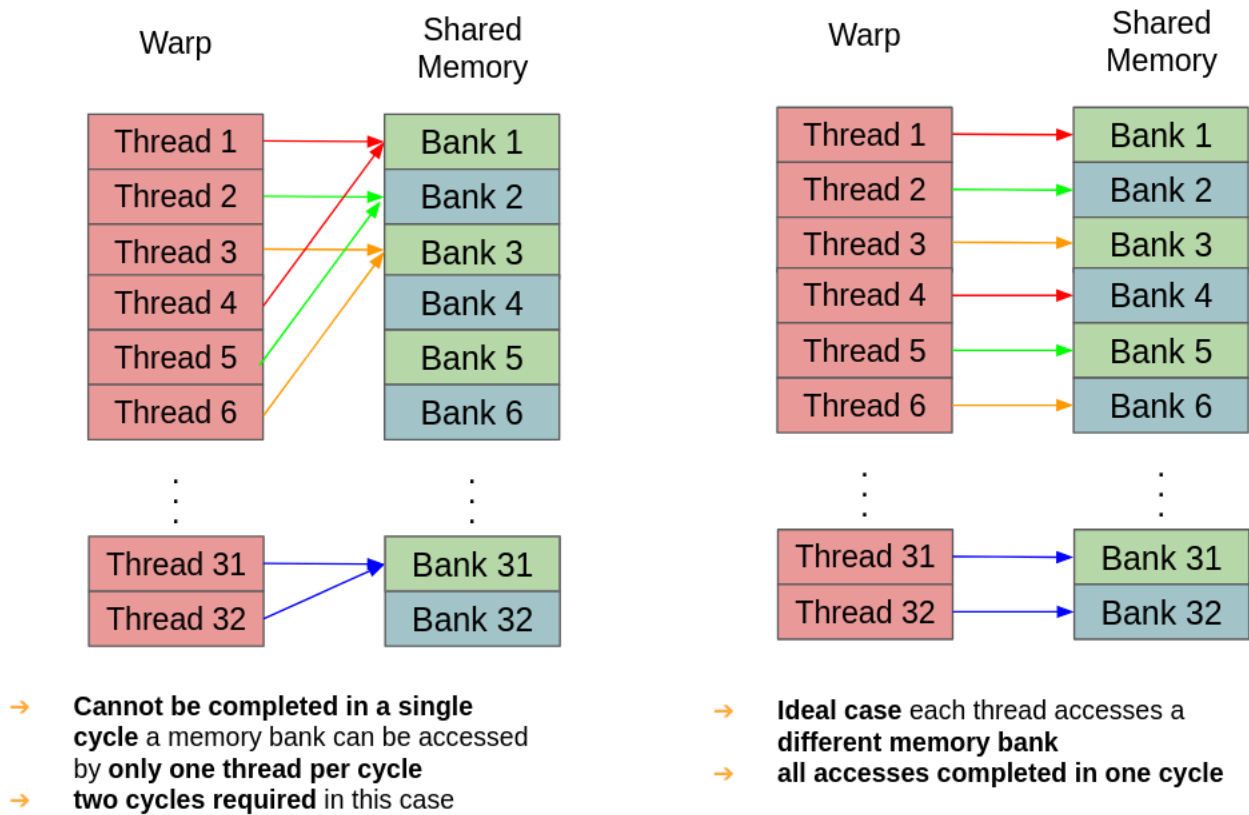


Figure 12. Illustration of Bank Conflict